

ExchangeLab

Electronic Exchange & Market Simulator

Freshman Technical Project Plan for Jane Street / JSIP

Project Overview

ExchangeLab is a simulated electronic financial exchange built around a real **limit-order-book matching engine**.

Rather than building a basic stock-trading interface or portfolio tracker, the project focuses on the underlying software infrastructure responsible for accepting orders, maintaining an order book, matching buyers and sellers, executing trades, and recording market activity.

A completed version of the project should support:

- Limit orders
- Market orders
- Price-time priority
- Partial fills
- Order cancellation
- Trade execution
- Trade history
- Simulated trading agents
- Performance benchmarking
- Automated tests
- Optional web dashboard

The project combines computer science and finance while emphasizing the areas most useful for software-engineering recruiting:

- Data structures
- Algorithms
- State management
- Object-oriented design
- Testing
- Performance
- System architecture
- Debugging
- Edge-case handling

Why This Project

The project is especially relevant to a student interested in both computer science and finance because it demonstrates technical depth without requiring advanced financial knowledge.

Instead of creating another AI stock predictor, financial chatbot, or CRUD portfolio tracker, ExchangeLab requires understanding how an actual trading system processes information.

Jane Street has publicly discussed exchange design and limit order books in its technical content, including concepts such as order priority, executions, cancellations, and market structure.

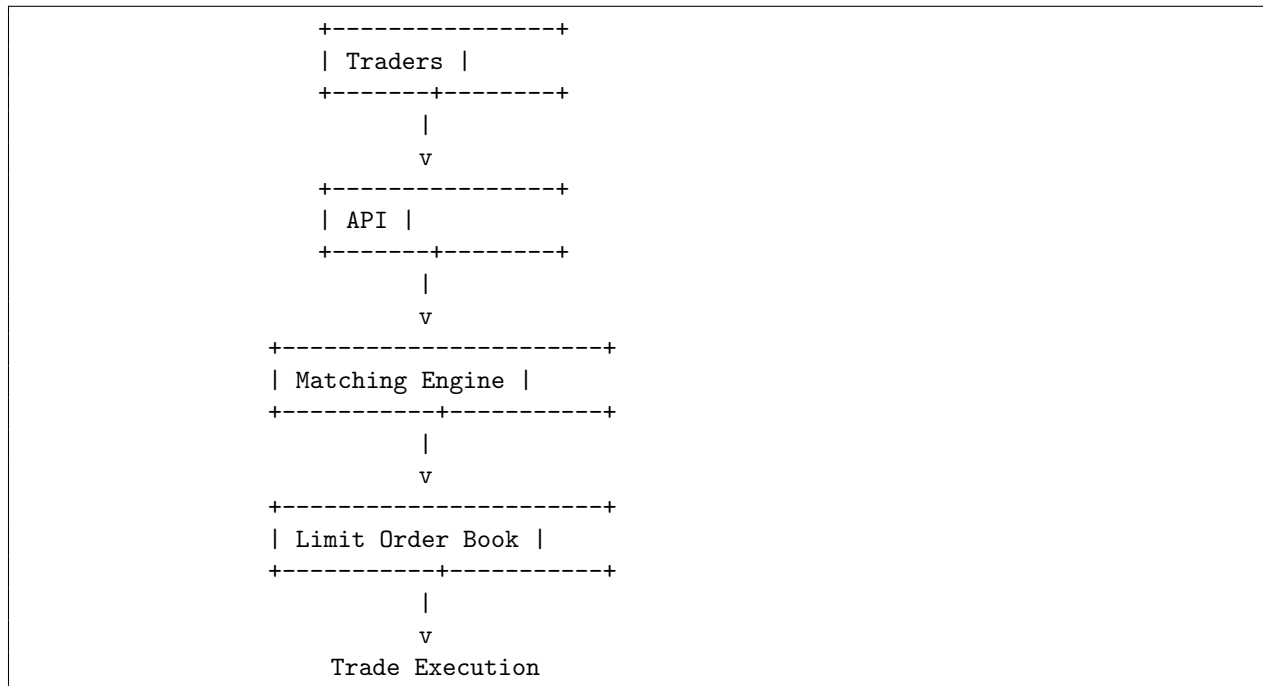
Jane Street also emphasizes fundamental programming ability, familiarity with data structures, communication, and reasoning in its software-engineering interview guidance.

Useful references:

- Jane Street — Building an Exchange: <https://www.janestreet.com/tech-talks/building-an-exchange/>
- Jane Street — Preparing for a Software Engineering Interview: <https://www.janestreet.com/preparing-for-a-software-engineering-interview/>
- Jane Street Immersion Program: <https://www.janestreet.com/join-jane-street/programs-and-events/jsip/>

Core System

The fundamental architecture should look approximately like this:



The **matching engine** is the most important component.

The frontend should be considered secondary.

A technically strong matching engine with an ugly command-line interface is preferable to a beautiful React application with a shallow backend.

Feature 1: Limit Orders

Users should be able to submit limit orders.

Example:

```
BUY 10 AAPL @ $200.00
```

or:

```
SELL 5 AAPL @ $201.00
```

A limit order specifies:

- Symbol
- Side: BUY or SELL
- Quantity
- Limit price
- Order ID
- Timestamp

A possible Python representation could be:

```
class Order:
    def __init__(self, order_id, symbol, side,
                  quantity, price, timestamp):
        self.order_id = order_id
        self.symbol = symbol
        self.side = side
        self.quantity = quantity
        self.price = price
        self.timestamp = timestamp
```

Feature 2: Limit Order Book

The system should maintain separate buy and sell sides of the market.

Example:

```
SELL

$203.00 30 shares
$202.00 15 shares
$201.00 20 shares

-----
```

```
$200.00 25 shares
$199.00 40 shares
$198.00 10 shares

BUY
```

The highest-priced buy order is the **best bid**.

The lowest-priced sell order is the **best ask**.

For example:

```
Best Bid: $200.00
Best Ask: $201.00
Spread: $1.00
```

Feature 3: Price-Time Priority

Orders should be matched according to **price-time priority**.

Price comes first.

If two orders have the same price, the earlier order receives priority.

Example:

```
Order A:
BUY 10 @ $200
Time: 10:01:00

Order B:
BUY 10 @ $200
Time: 10:01:05
```

Order A should receive execution priority because it arrived first.

This creates an important data-structure problem:

How can the system efficiently retrieve the best price while also preserving FIFO ordering among orders at the same price?

Feature 4: Market Orders

The exchange should also support market orders.

Example:

```
MARKET BUY 100 AAPL
```

A market order does not specify a price.

Instead, it immediately executes against the best available orders on the opposite side of the order book.

For example, suppose the sell side contains:

```
SELL 30 @ $100.00  
SELL 50 @ $100.05  
SELL 100 @ $100.10
```

A market order to buy 60 shares should execute:

```
30 shares @ $100.00  
30 shares @ $100.05
```

Feature 5: Partial Fills

Orders do not always execute completely.

Suppose the system receives:

```
BUY 100 @ $50.00
```

and encounters:

```
SELL 30 @ $49.00
```

The trade should execute 30 shares.

The original buy order should remain in the book with:

```
BUY 70 @ $50.00
```

The matching engine therefore needs to correctly update remaining quantities after every execution.

Feature 6: Order Cancellation

Users should be able to cancel an active order.

For example:

```
cancel_order(order_id)
```

An important performance question is how quickly an arbitrary order can be located and removed.

A naive implementation may require scanning a list:

```
O(n)
```

A more sophisticated implementation may maintain an additional hash map:

```
order_id -> order location
```

which can improve lookup performance.

This provides an excellent technical design decision to discuss in interviews.

Feature 7: Trade Execution and History

Every successful match should create a trade.

Example:

```
TRADE

Symbol: AAPL
Quantity: 50
Price: $200.25
Buyer ID: 1024
Seller ID: 8421
Timestamp: 14:32:19.428
```

Trade objects might include:

- Trade ID
- Buy order ID
- Sell order ID
- Symbol
- Quantity
- Execution price
- Timestamp

Trade history can later be used for visualization, analytics, and trading-bot performance.

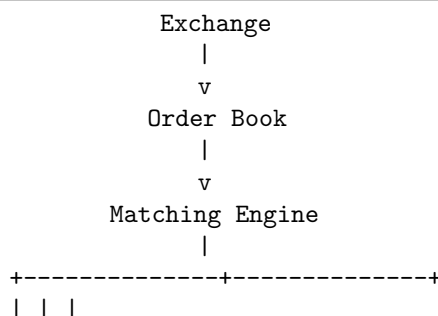
Trading Bots

Once the core exchange is complete, simulated traders can be added.

Possible bots include:

- Random Trader
- Market Maker
- Momentum Trader

The architecture could look like:



v v v
Random Bot Market Maker Momentum Bot

Random Trader

The simplest bot can randomly decide:

- Buy or sell
- Quantity
- Limit price
- Submission frequency

Its purpose is primarily to generate market activity.

Market-Making Bot

A simplified market-making bot could continuously maintain both a buy and sell order.

Example:

BUY @ \$99.95
SELL @ \$100.05

The bot attempts to earn the bid-ask spread while managing its inventory.

The program can track:

Cash: \$104,271
Inventory: -31 AAPL
PnL: +\$427
Trades: 1,284

The strategy can become progressively more sophisticated by changing quotes based on inventory.

For example, if the bot owns too many shares, it could lower both quotes to encourage selling inventory.

Momentum Trader

A basic momentum trader could observe recent price changes and submit orders in the direction of the trend.

This does not need to be a profitable real-world trading strategy.

Its purpose is to create interesting interactions between autonomous participants in the simulation.

Performance Benchmarking

One of the strongest additions to this project is a benchmarking system.

Generate large numbers of synthetic orders and measure how quickly the exchange processes them.

For example:

```
Orders processed: 1,000,000
Trades executed: 387,291
Average latency: 0.14 ms
Throughput: 85,000 orders/sec
```

Exact numbers are not important.

The important part is measuring real performance and understanding what affects it.

Version 1: Naive Implementation

The first version might use basic lists:

```
buy_orders = []
sell_orders = []
```

This makes the system easier to build but may create performance problems.

For example:

- Finding the best order may require scanning or sorting.
- Cancelling an arbitrary order may require $O(n)$ time.
- Repeated sorting may become expensive.

Version 2: Optimized Implementation

After benchmarking Version 1, redesign the order book using more appropriate data structures.

Possible tools include:

- Hash maps
- Priority queues
- Deques
- Sorted maps
- Balanced trees

For example:

```
price -> queue of orders
order_id -> order metadata
```

Orders at the same price can be stored in a FIFO queue.

A separate structure can efficiently identify the best available price.

This creates a strong engineering story:

My initial implementation used a simpler data structure, but cancellation and best-price retrieval became inefficient as order volume increased. I profiled the system and

redesigned the order book using separate price-level queues and indexed order lookup, reducing the complexity of several core operations.

Data Structures to Learn

The project naturally provides experience with:

- Arrays and lists
- Hash maps
- Sets
- Queues
- Deques
- Priority queues
- Heaps
- Trees
- Sorting
- Binary search

The exact implementation should evolve as technical knowledge improves.

It is perfectly acceptable to build a simpler system first and then optimize it.

In fact, that progression makes the project easier to explain because there is a clear engineering-development story.

Language Choice

The recommended first language is:

Python

Python makes it easier to focus on:

- Algorithms
- Data structures
- Architecture
- Testing
- Edge cases
- Correctness

rather than spending excessive time dealing with language-specific complexity.

A later extension could rewrite the matching engine in:

C++

The project could then benchmark:

Python Matching Engine vs. C++ Matching Engine
--

This would provide an opportunity to explore:

- Memory management
- Static typing
- Compilation
- Performance
- Lower-level optimization

However, C++ should be considered an extension rather than a requirement.

The Python implementation should first be correct, tested, and well understood.

Recommended Repository Structure

A possible GitHub repository structure is:

```
ExchangeLab/  
|  
|-- engine/  
| |-- order.py  
| |-- order_book.py  
| |-- matching_engine.py  
| |-- trade.py  
|  
|-- strategies/  
| |-- random_trader.py  
| |-- market_maker.py  
| |-- momentum.py  
|  
|-- simulation/  
| |-- simulator.py  
| |-- market.py  
|  
|-- api/  
| |-- server.py  
|  
|-- tests/  
| |-- test_matching.py  
| |-- test_orders.py  
| |-- test_cancellation.py  
|  
|-- benchmarks/  
| |-- benchmark.py  
|  
|-- frontend/  
|
```

```
|-- README.md  
|  
|-- requirements.txt
```

This architecture separates:

- Exchange logic
- Trading strategies
- Simulation infrastructure
- API
- Testing
- Performance benchmarking
- Frontend

Testing

Testing should be treated as a major component of the project.

Important tests include:

- Buy order does not match another buy order.
- Sell order does not match another sell order.
- Crossing buy and sell orders execute.
- Orders at the same price respect time priority.
- Partial fills leave the correct remaining quantity.
- Fully filled orders disappear from the book.
- Cancelled orders cannot execute.
- Market orders consume multiple price levels correctly.
- Empty order books do not cause crashes.
- Invalid quantities are rejected.
- Duplicate order IDs are rejected.

Tests are valuable not only because they improve reliability, but because they demonstrate that the project was engineered rather than simply assembled.

Frontend

The frontend should be built **after** the backend is working.

Possible dashboard components include:

- Live order book

- Best bid
- Best ask
- Bid-ask spread
- Recent trades
- Price chart
- Trading-bot inventory
- Trading-bot PnL
- Orders per second
- Execution latency

A possible interface could display:

```
AAPL Last: $201.42
```

```
ORDER BOOK
```

```
SELL
```

```
$201.50 200
```

```
$201.47 150
```

```
$201.45 75
```

```
-----
```

```
$201.40 120
```

```
$201.35 300
```

```
$201.30 80
```

```
BUY
```

```
Recent Trades
```

```
100 @ $201.42
```

```
25 @ $201.40
```

```
75 @ $201.39
```

The frontend can be created using a framework such as React, Next.js, or another familiar web stack.

However, frontend complexity should not replace backend depth.

Using AI and Vibe Coding

AI-assisted development is acceptable and can substantially speed up development.

However, the project should not become something that the developer cannot explain.

A useful rule is:

If it is on the resume, the important parts should be understandable well enough to rebuild them without AI assistance.

This does not mean every line must be memorized.

It means understanding:

- Why each major data structure was chosen
- How orders are matched
- How price-time priority works
- How partial fills work
- How cancellations work
- What the major time complexities are
- What important bugs occurred
- How those bugs were fixed
- Which architectural decisions were made
- What could be improved in a future version

Good Uses of AI

AI can be used aggressively for:

- Frontend development
- CSS
- API boilerplate
- Documentation
- Test scaffolding
- Generating additional edge cases
- Debugging
- Refactoring
- Explaining unfamiliar concepts
- Reviewing code

Core Components to Understand Personally

Extra attention should be given to personally understanding:

```
Order Representation
    |
    v
Order Book
    |
    v
Matching Algorithm
    |
```

```
      v
Execution Logic
      |
      v
Data Structures
```

AI can still assist with these components.

However, after using AI, the developer should study and modify the generated code until the implementation and design decisions are fully understood.

Questions You Should Be Able to Answer

Before placing ExchangeLab on a resume, you should be able to answer questions such as:

1. Why did you choose your current data structure for the order book?
2. What is the time complexity of inserting an order?
3. What is the time complexity of cancelling an order?
4. How do you find the best bid and best ask?
5. How do you preserve FIFO ordering at the same price?
6. How does your program handle partial fills?
7. What happens when a market order is larger than available liquidity?
8. How do you prevent duplicate order IDs?
9. What was the hardest bug you encountered?
10. What was your first implementation?
11. What did you later optimize?
12. What would break if the exchange received 100 times more traffic?
13. What would you redesign if you had another month?
14. Why did you choose Python?
15. How would the architecture change if you rewrote the engine in C++?

If these questions can be answered confidently, AI assistance during development should not be a problem.

Recommended Development Roadmap

Week 1 — Orders and Basic Order Book

Goals:

- Learn basic limit-order-book concepts.
- Create the Order class.

- Create buy and sell sides.
- Add orders.
- Display the order book.
- Determine best bid and best ask.

Do not build the frontend.

Focus exclusively on correctness.

Week 2 — Matching Engine

Implement:

- Price matching
- Price-time priority
- Full fills
- Partial fills
- Market orders
- Order cancellation
- Trade creation

At the end of Week 2, the exchange should function through a command-line interface.

Week 3 — Testing and Edge Cases

Build a comprehensive automated test suite.

Test:

- Matching behavior
- Priority behavior
- Partial fills
- Cancellations
- Market orders
- Invalid input
- Empty books
- Multiple price levels

Fix correctness problems before expanding the project.

Week 4 — Market Simulation

Add simulated participants.

Start with:

- Random trader
- Basic market maker

If time permits, add:

- Momentum trader

Track:

- Cash
- Inventory
- Trades
- Profit and loss

Week 5 — Benchmarking and Optimization

Generate:

- 10,000 orders
- 100,000 orders
- 1,000,000 orders

Measure:

- Throughput
- Average latency
- Matching time
- Cancellation time

Identify bottlenecks and improve the data structures.

Document the difference between the original implementation and optimized version.

Week 6 — Frontend and GitHub Polish

Only now create the interface.

Add:

- Order-book visualization
- Recent trades
- Bot statistics
- Charts

- Performance statistics

Then polish the GitHub repository.

README Structure

The final README should contain:

1. Project description
2. Demo screenshot
3. Architecture diagram
4. Features
5. Installation instructions
6. Usage
7. Matching-engine explanation
8. Data structures
9. Testing
10. Benchmarks
11. Design decisions
12. Known limitations
13. Future improvements

Example GitHub Description

A concise GitHub description could be:

ExchangeLab is a simulated electronic exchange implementing a price-time-priority limit order book, market and limit orders, partial fills, order cancellation, trade execution, algorithmic trading agents, and performance benchmarking.

Potential Resume Bullet

After the project is complete and the performance metrics are real, a resume entry could look approximately like:

ExchangeLab — Electronic Exchange Simulator

Built a simulated electronic exchange implementing a price-time-priority limit order book, market and limit orders, partial fills, cancellations, and automated trade execution; developed algorithmic trading agents and benchmarked matching-engine performance across large synthetic workloads.

A stronger version can eventually include measured performance.

For example:

Built a price-time-priority limit-order-book matching engine supporting market and limit orders, partial fills, cancellations, and automated execution; optimized core data structures to process X orders/sec across simulated workloads of Y+ orders.

The values X and Y should only be added after they have actually been measured.

What Not to Build Instead

For the goal of demonstrating technical depth, avoid spending the entire semester on:

- Generic stock portfolio tracker
- AI stock predictor
- ChatGPT finance chatbot
- Basic investment dashboard
- CRUD stock website
- Another thin wrapper around an LLM API

These projects can look polished but often provide less opportunity to discuss algorithms, data structures, systems design, and performance.

ExchangeLab provides much more technical material to discuss.

Primary Objectives

The project should optimize for the following priorities:

1. **Correctness**

The exchange should behave predictably and correctly.

2. **Understanding**

The developer should understand the important implementation details.

3. **Technical Depth**

The project should require meaningful algorithms and data structures.

4. **Testing**

Important behaviors should have automated tests.

5. **Performance**

The system should eventually be benchmarked and optimized.

6. **Documentation**

Design decisions should be explained clearly.

7. Presentation

A polished frontend and README should come only after the technical foundation is complete.

Final Target

By the end of the project, ExchangeLab should demonstrate the ability to:

- Design a non-trivial software system
- Select appropriate data structures
- Implement algorithms
- Handle complex mutable state
- Think about computational complexity
- Write automated tests
- Debug edge cases
- Measure performance
- Optimize software
- Explain engineering tradeoffs
- Apply computer science to financial markets

The most important goal is not simply having the project listed on a resume.

The goal is to reach the point where an interviewer can spend twenty minutes asking technical questions about ExchangeLab and every question creates an opportunity to demonstrate genuine computer-science knowledge.